

TRAINING REFERENCE

Databricks Data Engineer Associate

Reference guide for the three-day course

Contents

MODULE 01	Platform and workspace	3
MODULE 02	Data ingestion	5
MODULE 02A	Lakeflow Connect	7
MODULE 03	Delta Lake	9
MODULE 05A	Transformations and modeling	11
MODULE 04	Delta optimization and performance	14
MODULE 05	Incremental ingestion	16
MODULE 06	Medallion architecture	18
MODULE 07	Lakeflow Spark Declarative Pipelines	19
MODULE 08	Lakeflow Jobs	22
MODULE 09	CI/CD with Git folders and bundles	24
WORKSHOP	Troubleshooting and monitoring	26
MODULE 10	Governance and security	28
APPENDIX A	Exam at a glance	31

MODULE 01

Platform and workspace

Platform components

Component	Role
Delta Lake	Open table format: Parquet files plus a transaction log; ACID, time travel, schema enforcement
Unity Catalog	Governance of data and AI assets: privileges, lineage, audit, discovery
Lakeflow Connect, Spark Declarative Pipelines, Jobs	Ingestion connectors; declarative ETL (formerly Delta Live Tables); orchestration (formerly Workflows)
Git folders, Declarative Automation Bundles	Version control (formerly Repos); CI/CD as code (formerly Databricks Asset Bundles)

```

Metastore          one per region, attached to workspaces
├─ Storage credential, external location, connection
├─ Catalog          e.g. retailhub_jan_kowalski
│   └─ Schema       e.g. bronze, silver, gold
│       └─ Table, view, materialized view, streaming table, volume, function, model

```

Objects use three-part names, `catalog.schema.object`. Files live in volumes, `/Volumes/<catalog>/<schema>/<volume>/path`; DBFS root and mounts are legacy.

Compute and cost models

Compute	Billing	Best for	Limitations
Serverless notebooks and jobs	DBUs per second of use; infrastructure included; starts in seconds	Default for notebooks and ETL jobs	Few Spark confs, no init scripts, GPUs or Spark UI
All-purpose cluster (classic)	DBUs plus cloud VMs while running	Interactive work needing custom setup	Idle time billed; set auto-termination
Job cluster (classic)	Lower jobs DBU rate plus VMs, only during the run	Scheduled jobs needing init scripts, JARs, GPUs, spot	Created per run
SQL warehouse (serverless, pro, classic)	DBUs by size while running; auto-stop	SQL editor, dashboards, BI tools over JDBC/ODBC, many concurrent queries	SQL only
Serverless pipelines	DBUs for pipeline updates	Lakeflow Spark Declarative Pipelines	Pipeline code only

Cost levers on classic compute: autoscaling, auto-termination, spot workers, cluster policies, Photon. Access modes: **standard** (formerly shared, multi-user) and **dedicated** (formerly single user).

Serverless: what changes

Classic habit	On serverless
Spark UI	Query profile
processingTime or continuous trigger, memory or console sink	trigger(availableNow=True) into a Delta table
Setting any Spark conf	Only a few, e.g. spark.sql.shuffle.partitions (default auto), spark.sql.session.timeZone, spark.sql.ansi.enabled
DBFS root, /tmp	Volumes or workspace files
Cluster libraries, init scripts	%pip install, notebook or job environments
RDD API, cache(), global temp views, Scala or R notebooks	Not supported

MODULE 02

Data ingestion

Ingestion patterns

Pattern	Behaviour	Tools
Full batch	Reads everything, overwrites	<code>spark.read</code> , <code>read_files</code> , CTAS
Incremental batch	Processes only new data, then stops	<code>COPY INTO</code> , Auto Loader with <code>availableNow</code> , managed connectors
Streaming	Runs continuously	Auto Loader, Kafka, streaming tables

Reading files

```
from pyspark.sql.types import StructType, StructField, StringType, DateType

# CSV with a user schema is matched BY POSITION: list every column in file order
customers_schema = StructType([
    StructField("customer_id", StringType(), False), StructField("first_name", StringType()),
    StructField("last_name", StringType()),           StructField("email", StringType()),
    StructField("phone", StringType()),               StructField("city", StringType()),
    StructField("state", StringType()),               StructField("country", StringType()),
    StructField("registration_date", DateType()),     StructField("customer_segment", StringType()),
])
customers = (spark.read.format("csv").option("header", "true")
            .schema(customers_schema)                  # one pass, no inference
            .load("/Volumes/<catalog>/default/datasets/customers/customers.csv"))

orders = spark.read.format("json").load("../orders/orders_batch.json") # JSON: BY NAME
products = spark.read.parquet("../products/products.parquet")          # schema in the file
```

Option	Meaning
<code>header</code> , <code>inferSchema</code>	Header row; detect types at the cost of an extra pass
<code>sep</code> , <code>nullValue</code> , <code>dateFormat</code>	CSV parsing and conversion
<code>multiLine</code>	A record spans several lines (pretty-printed JSON, quoted CSV newlines)
<code>mode</code>	PERMISSIVE (default; raw row in <code>_corrupt_record</code>), DROPMALFORMED, FAILFAST

With `header=true` and a user schema, CSV header names are ignored (`enforceSchema` is true by default). A schema in the wrong order loads values into the wrong columns without an error. In JSON, a schema field name that does not exist in the file is silently NULL.

read_files and CTAS

```
CREATE OR REPLACE TABLE bronze.customers AS      -- CTAS: snapshot, a rerun rewrites everything
SELECT *, _metadata.file_path AS source_file, current_timestamp() AS _load_ts
FROM read_files('/Volumes/<catalog>/default/datasets/customers/customers.csv',
               format => 'csv', header => true, schemaHints => 'registration_date DATE');
```

`read_files` infers the schema, puts non-matching values into `_rescued_data`, and reads csv, json, parquet, avro, orc, text, binaryFile, xml. File metadata is in `_metadata.file_path`, `file_name`, `file_size`, `file_modification_time`; `input_file_name()` is not supported in Unity Catalog.

```
(df.write.mode("append")                # append | overwrite | error (default) | ignore
  .option("mergeSchema", "true")        # add new columns; overwriteSchema replaces the schema
  .saveAsTable("catalog.bronze.orders"))
```

Local files: Add data, then **Create or modify table**, uploads CSV, JSON, Parquet, Avro or text files into a Delta table; or upload them to a volume and use `read_files`.

JDBC and REST sources

```
jdbc_df = (spark.read.format("jdbc")
  .option("url", "jdbc:sqlserver://host:1433;database=sales")
  .option("dbtable", "dbo.orders")                # or .option("query", "SELECT ...")
  .option("user", dbutils.secrets.get("jdbc", "user"))
  .option("password", dbutils.secrets.get("jdbc", "password"))
  .option("partitionColumn", "order_id").option("lowerBound", 1)
  .option("upperBound", 1_000_000).option("numPartitions", 8)
  .load())
jdbc_df.write.mode("append").saveAsTable("catalog.bronze.orders_jdbc")

import requests
resp = requests.get("https://api.example.com/rates", timeout=30); resp.raise_for_status()
dbutils.fs.put(f"/Volumes/{catalog}/bronze/landing/rates/{run_date}.json", resp.text, True)
```

Credentials always come from secrets. JDBC reads are full reads unless you filter on a watermark column yourself. Land REST responses as raw files, load them with `read_files` or Auto Loader, and schedule the notebook as a Lakeflow Jobs task with dates passed as job parameters.

MODULE 02A

Lakeflow Connect

Choosing an ingestion method

Source	First choice
Files arriving in cloud storage or a volume	Auto Loader (standard connector), or <code>read_files</code> in a streaming table
Kafka, Kinesis, Pub/Sub, Pulsar	Standard connector in Structured Streaming or a pipeline
Supported SaaS app (Salesforce, Workday, ServiceNow)	Managed connector: no code, incremental, governed
Supported database (SQL Server, Azure SQL, MySQL, PostgreSQL, Oracle)	Managed connector with CDC or change tracking
Source without a Databricks connector	Partner tool (Fivetran, Qlik, Informatica) through Partner Connect
One-off extract, small lookup table	JDBC or REST in a notebook; you own incrementality, retries and schema drift

Criterion	Points to
High volume, millions of files, frequent arrivals	Auto Loader with file events
Low-latency changes from an operational database	Managed database connector (CDC)
Semi-structured or changing schema	Auto Loader or <code>read_files</code> with <code>_rescued_data</code> , VARIANT columns
Central governance, no credentials in code	Managed connector: the connection is a Unity Catalog object
Little engineering capacity	Managed or partner connector before custom code

Standard connectors (cloud storage, SFTP, message buses) are configured in code and write to any table. **Managed connectors** (SaaS apps, databases) are configured in the UI, API or bundles and maintain streaming tables through an ingestion pipeline.

Managed database connector

```
Source database (CDC or change tracking on)
-> 1. Connection          UC object: host, port, credentials
-> 2. Ingestion gateway   classic compute, runs continuously, extracts snapshots and changes
-> Staging volume        UC volume with extracted data
-> 3. Ingestion pipeline  serverless, run on a schedule by a job, applies changes
-> catalog.schema.table  streaming tables, SCD type 1 or 2
```

SaaS connectors need no gateway. The gateway must keep running so changes are read before the source truncates its change log.

Prerequisite	Detail
Source	CDC or change tracking on the tables; a low-privilege login
Privileges	CREATE CONNECTION on the metastore; USE CATALOG, USE SCHEMA, CREATE TABLE, CREATE VOLUME on staging and target
History	SCD_TYPE_1 (overwrite) or SCD_TYPE_2 (history; requires CDC)

Semi-structured data

Need	PySpark	SQL
Nested field, flatten struct	<code>col("address.city"), select("item.*")</code>	<code>address.city, item.*</code>
Parse or query a JSON string	<code>from_json(col("raw"), schema)</code>	<code>from_json(raw, 'city STRING'), raw:a.b</code>
Flexible storage	<code>parse_json(col("raw")) into VARIANT</code>	<code>parse_json(raw), then v:customer.city::string</code>
Array to rows	<code>explode("items"), explode_outer keeps empty arrays</code>	<code>explode(items)</code>

MODULE 03

Delta Lake

A Delta table is Parquet data files plus a transaction log in `_delta_log/`; each commit adds a log entry and readers see only committed versions. Tables created in Databricks are Delta by default.

Changing data

```
INSERT INTO silver.customers SELECT * FROM staged_customers;
UPDATE silver.customers SET customer_segment = 'Premium' WHERE customer_id = 'CUST000042';
DELETE FROM silver.customers WHERE customer_id IS NULL;

MERGE INTO silver.customers AS t
USING customer_updates AS s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.is_deleted THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;      -- optional: target rows absent from the source
```

MERGE fails when one target row matches several source rows: deduplicate the source on the key first.

SCD type 2 with MERGE: staged union

A single MERGE cannot close the current row and insert its new version for the same key, so changed rows are staged twice: with the real key (closes the old row) and with a NULL key (never matches, so it is inserted).

```
MERGE INTO gold.dim_customer AS t
USING (
  SELECT s.customer_id AS merge_key, s.* FROM updates s
  UNION ALL
  SELECT NULL AS merge_key, s.*
  FROM updates s JOIN gold.dim_customer d ON s.customer_id = d.customer_id AND d.is_current
  WHERE s.email <> d.email OR s.city <> d.city
) AS m
ON t.customer_id = m.merge_key AND t.is_current
WHEN MATCHED AND (t.email <> m.email OR t.city <> m.city) THEN
  UPDATE SET is_current = false, end_date = current_date()
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, city, is_current, start_date, end_date)
  VALUES (m.customer_id, m.email, m.city, true, current_date(), NULL);
```

In a pipeline, prefer `AUTO CDC ... STORED AS SCD TYPE 2` (Module 07).

Schema, history and time travel

Mechanism	Effect
Schema enforcement (default)	Writes with unknown columns or incompatible types fail
mergeSchema / overwriteSchema	Add new columns on append / replace the schema on overwrite
MERGE WITH SCHEMA EVOLUTION INTO	MERGE may add source columns
CHECK constraint, NOT NULL, GENERATED ALWAYS AS IDENTITY	Quality rules and surrogate keys at table level

```
DESCRIBE HISTORY silver.customers;      -- version, timestamp, operation, user, metrics
DESCRIBE DETAIL silver.customers;      -- location, numFiles, sizeInBytes, clustering
SELECT * FROM silver.customers VERSION AS OF 3;
SELECT * FROM silver.customers TIMESTAMP AS OF '2026-09-01 08:00:00';
RESTORE TABLE silver.customers TO VERSION AS OF 3;      -- itself a new version
VACUUM silver.customers DRY RUN;      -- then VACUUM t [RETAIN 240 HOURS]
```

Table property	Default	Governs
delta.deletedFileRetentionDuration	7 days	How long removed files are kept before VACUUM deletes them, so how far back time travel works
delta.logRetentionDuration	30 days	How long history entries are kept

A retention below 7 days is blocked by a safety check. After VACUUM, versions that need deleted files can no longer be read.

Managed and external tables, including conversion between them, are covered in Module 10.

Common Delta errors

Error	Fix
Multiple source rows matched in MERGE	Deduplicate the source on the merge key
ConcurrentAppendException	Make concurrent operations disjoint with explicit predicates on partition or clustering columns, rely on row-level concurrency (liquid clustering with deletion vectors), or retry. A stricter isolation level does not fix it
File not found on an old version	Read a retained version; increase delta.deletedFileRetentionDuration for the future

MODULE 05A

Transformations and modeling

Cleaning bronze into silver

```
from pyspark.sql import functions as F

silver = (spark.read.table("bronze.orders")
    .filter(F.col("order_id").isNotNull() & F.col("customer_id").isNotNull())
    .withColumn("order_ts", F.to_timestamp("order_datetime"))
    .withColumn("payment_method", F.initcap(F.trim("payment_method")))
    .withColumn("quantity", F.col("quantity").cast("int"))
    .fillna({"discount_percent": 0})
    .dropDuplicates(["order_id"]))
silver.write.mode("overwrite").saveAsTable("silver.orders")
```

Operation	PySpark	SQL
Drop or replace nulls	dropna(subset=["id"]), fillna({"city": "Unknown"})	WHERE id IS NOT NULL, COALESCE(city, 'Unknown')
Add, rename, drop columns	withColumn, withColumnRenamed, drop("c1", "c2")	expr AS new, old AS new, SELECT * EXCEPT (c1, c2)
Cast, safe cast	col("x").cast("int"), F.expr("try_cast(x AS INT)")	CAST(x AS INT), TRY_CAST(x AS INT)
Split a string	F.split("full_name", " ")[0]	split(full_name, ' ')[0]
Filter	filter(cond) = where(cond)	WHERE
Explode an array	F.explode("items"), explode_outer, posexplode	explode(items)
Conditional value	F.when(c, v).otherwise(d)	CASE WHEN c THEN v ELSE d END

Joins and unions

```
orders.join(customers, "customer_id", "inner")           # one key column in the result
orders.join(customers, ["customer_id", "country"], "left") # multiple keys
orders.join(customers, (orders.cust == customers.customer_id) &
              (orders.region == customers.region), "left")
orders.join(F.broadcast(stores), "store_id")             # force a broadcast join
sizes.crossJoin(colors)                                  # every combination
```

Join type	Returns
inner / left / right / full	Matches only / all left rows / all right rows / all rows, NULLs where no match
left_semi / left_anti	Left rows with / without a match, left columns only
cross	Cartesian product

A **broadcast join** sends the small table to every executor, so the large table is not shuffled. Spark chooses it automatically below `spark.sql.autoBroadcastJoinThreshold` (10 MB); force it with `broadcast(df)` or `/*+ BROADCAST(s) */`.

PySpark	SQL	Columns matched	Duplicates
<code>a.union(b)</code> , alias <code>unionAll</code>	<code>UNION ALL</code>	By position	Kept
<code>a.union(b).distinct()</code>	<code>UNION</code>	By position	Removed
<code>a.unionByName(b, allowMissingColumns=True)</code>	<code>UNION ALL</code> with columns listed in the same order	By name; missing become NULL	Kept

Deduplication and aggregates

```
from pyspark.sql.window import Window
w = Window.partitionBy("customer_id").orderBy(F.col("_load_ts").desc())
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")

orders.groupBy("payment_method").agg(
    F.count("*").alias("orders"),
    F.countDistinct("customer_id").alias("customers_exact"),
    F.approx_count_distinct("customer_id").alias("customers_approx"), # HyperLogLog++, rsd 0.05
    F.mean("total_amount").alias("avg_amount"))
orders.select("total_amount").summary().show() # count, mean, stddev, min, quartiles, max
```

SQL deduplication: `SELECT * FROM t QUALIFY row_number() OVER (PARTITION BY id ORDER BY ts DESC) =`

1. Ranking on ties: `row_number` 1, 2, 3; `rank` 1, 2, 2, 4; `dense_rank` 1, 2, 2, 3. `lag` and `lead` return the previous and next value in the window.

The four tuning parameters

Parameter	Default	Controls	Set
<code>spark.sql.shuffle.partitions</code>	200 (classic), auto (serverless)	Partitions after joins and aggregations	<code>spark.conf.set</code> or <code>SET</code> , also on serverless
<code>spark.default.parallelism</code>	Total cores	Partitions for RDD operations	Cluster Spark config
<code>spark.executor.memory</code> , <code>spark.driver.memory</code>	From node type	JVM heap per executor or driver	Cluster config at creation; not at runtime, not on serverless
<code>spark.sql.autoBroadcastJoinThreshold</code>	10 MB	Largest table broadcast automatically; -1 disables	<code>spark.conf.set</code> on classic compute

Re-measure after every single change: run the same action (for example `df.write.format("noop").mode("overwrite").save()`), compare wall-clock time and the plan in the Spark UI or query profile. Python UDFs are slow (row-by-row serialization); prefer built-in functions, SQL UDFs or pandas UDFs.

MODULE 04

Delta optimization and performance

Data layout

Technique	Syntax	Use
Compaction	<code>OPTIMIZE t</code>	Many small files
Liquid clustering	<code>CLUSTER BY (c1, c2), then OPTIMIZE t</code>	Default for new tables; keys can change without rewriting data
Automatic clustering	<code>CLUSTER BY AUTO</code>	Keys chosen from query history
Partitioning, Z-order (legacy)	<code>PARTITIONED BY (col), OPTIMIZE t</code> <code>ZORDER BY (col)</code>	Older tables; not combinable with liquid clustering

```
CREATE TABLE silver.sales (sale_id STRING, customer_id STRING, region STRING, amount DOUBLE)
CLUSTER BY (customer_id, region);
ALTER TABLE silver.sales CLUSTER BY AUTO;      -- or CLUSTER BY (col), CLUSTER BY NONE
OPTIMIZE silver.sales;                          -- incremental; OPTIMIZE ... FULL reclusters all
```

Liquid clustering: up to 4 keys; data is clustered on write above size thresholds, by `OPTIMIZE`, and by predictive optimization. `CLUSTER BY AUTO` requires a Unity Catalog managed table, predictive optimization and DBR 15.4 LTS or later. In Python: `df.write.clusterBy("customer_id").saveAsTable("t")`.

Predictive optimization and deletion vectors

Predictive optimization runs `OPTIMIZE`, `VACUUM` and `ANALYZE` automatically on Unity Catalog **managed** tables. It is on by default for accounts created from 11 November 2024 and is being rolled out to older accounts.

```
ALTER SCHEMA retailhub.staging DISABLE PREDICTIVE OPTIMIZATION;  -- ENABLE | DISABLE | INHERIT
SELECT * FROM system.storage.predictive_optimization_operations_history;
```

Deletion vectors (default for new tables) make `DELETE`, `UPDATE` and `MERGE` mark rows as removed instead of rewriting Parquet files; `OPTIMIZE` or `REORG TABLE t APPLY (PURGE)` rewrites the files later.

Skew, shuffle and spill

Problem	Signature in Spark UI or query profile	Fix
Skew	Max task duration and input size far above the median in one stage	AQE skew join (on by default); handle the hot key separately; salt aggregation keys
Heavy shuffle	Large shuffle read and write; <code>SortMergeJoin</code> with a small side	Broadcast the small table; filter and prune columns before joins
Spill	Spill (memory) and spill (disk) above 0	More, smaller shuffle partitions; less skew; more memory per task
Tiny tasks	Thousands of millisecond tasks in a shuffle stage	Fewer shuffle partitions or auto
Row explosion	Join output rows far above input rows	Fix the join key

Adaptive query execution (AQE) re-plans at run time: it coalesces small shuffle partitions, switches to broadcast joins and splits skewed partitions. Check plans with `df.explain(mode="formatted")`: `BroadcastHashJoin`, `SortMergeJoin`, `Exchange (shuffle)`, `PushedFilters`. `repartition(n)` shuffles fully; `coalesce(n)` merges partitions without a full shuffle.

MODULE 05

Incremental ingestion

COPY INTO

```
CREATE TABLE IF NOT EXISTS bronze.customers_batch;           -- schemaless target

COPY INTO bronze.customers_batch
FROM (SELECT *, _metadata.file_path AS source_file
      FROM '/Volumes/retailhub/bronze/landing/customers/')
FILEFORMAT = CSV
PATTERN = '*.csv'
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');                       -- needed for a schemaless target
```

Behaviour	Detail
Idempotent	Already loaded files are skipped on rerun, even if modified
Reload	<code>COPY_OPTIONS ('force' = 'true')</code>
Select files, dry run	<code>FILES = ('a.csv')</code> or <code>PATTERN; VALIDATE 10 ROWS</code>
Scale	Thousands of files per run; for millions or continuous arrival use Auto Loader

Auto Loader

```
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", f"/Volumes/{catalog}/bronze/checkpoints/orders_schema")
  .option("cloudFiles.inferColumnTypes", "true")
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
  .load(f"/Volumes/{catalog}/bronze/landing/orders/")
  .select("?", F.col("_metadata.file_path").alias("source_file"))
  .writeStream
  .option("checkpointLocation", f"/Volumes/{catalog}/bronze/checkpoints/orders")
  .trigger(availableNow=True)           # everything new, then stop: incremental batch
  .toTable(f"{catalog}.bronze.orders_autoloader"))
```

Other options: `cloudFiles.schemaHints` (force column types), `cloudFiles.maxFilesPerTrigger`, `cloudFiles.includeExistingFiles`.

File detection mode	How files are found	Use
Directory listing (default)	Lists the input path	Simple start, moderate directories
File notification with file events (recommended)	<code>cloudFiles.useManagedFileEvents = true</code> ; events managed on the UC external location (file events enabled; DBR 14.3 LTS+)	Large or fast-growing directories
File notification, legacy	<code>cloudFiles.useNotifications = true</code> ; Auto Loader creates cloud queues	Existing streams

<code>schemaEvolutionMode</code>	On a new column
<code>addNewColumns</code> (default without a schema)	Stream fails once and updates the schema; the restart includes the column
<code>rescue</code>	Schema unchanged; unknown columns and bad values go to <code>_rescued_data</code>
<code>failOnNewColumns</code>	Stream fails until fixed
<code>none</code> (default with a provided schema)	New columns ignored

Streaming output modes: **append** (new rows), **update** (changed aggregates), **complete** (whole result). One checkpoint per query; never share it.

MODULE 06

Medallion architecture

Layer	Content	Writes	Quality
Bronze	Raw data plus <code>_load_ts</code> , <code>source_file</code>	Append	None: keep everything, replayable
Silver	Typed, cleaned, deduplicated, conformed; SCD1 or SCD2 dimensions	MERGE or overwrite	Rules enforced, rejects quarantined
Gold	Aggregates, facts and dimensions, KPIs	Overwrite or materialized views	Trusted, business names

SCD type	Behaviour	Question signal
1	Overwrite, no history	Only current values matter
2	New row per change with validity columns	"Full history", "value at the time of the order"

Data quality checks

```
ALTER TABLE silver.orders ADD CONSTRAINT valid_qty CHECK (quantity > 0); -- bad writes fail
```

```
rules = "order_id IS NOT NULL AND customer_id IS NOT NULL AND quantity > 0"
bronze.filter(rules).write.mode("append").saveAsTable("silver.orders")
bronze.filter(f"NOT ({rules}) OR ({rules}) IS NULL") \
    .write.mode("append").saveAsTable("silver.orders_quarantine") # NULL-safe inverse
```

Inside a pipeline the same rules are expectations (Module 07).

Gold objects in Unity Catalog

Object	Stores data	Freshness	Created with
Table	Yes	When a job writes it	CTAS, <code>saveAsTable</code> , MERGE
View	No; query runs at read time	Always current	CREATE VIEW
Materialized view	Yes, precomputed	On refresh, incremental when possible	Pipeline, or CREATE MATERIALIZED VIEW ... SCHEDULE EVERY 1 HOUR on a serverless or pro SQL warehouse
Streaming table	Yes, append-driven	Each refresh reads only new input	Pipeline, or CREATE STREAMING TABLE ... AS SELECT * FROM STREAM read_files(...)

MODULE 07

Lakeflow Spark Declarative Pipelines

A pipeline is a set of SQL and Python files that declare datasets. The engine builds the dependency graph, runs updates in order, keeps checkpoints, retries, enforces expectations and writes an event log. Datasets in the same pipeline are referenced by name, without the old `LIVE.` prefix.

Dataset	SQL	Python (from <code>pyspark import pipelines as dp</code>)	Processing
Streaming table	<code>CREATE OR REFRESH STREAMING TABLE</code>	<code>@dp.table</code> returning a streaming <code>DataFrame</code>	Reads only new input: <code>FROM STREAM(src)</code>
Materialized view	<code>CREATE OR REFRESH MATERIALIZED VIEW</code>	<code>@dp.materialized_view</code>	Keeps the query result correct: <code>FROM src</code>
Temporary view	<code>CREATE TEMPORARY VIEW</code>	<code>@dp.temporary_view</code>	Not stored, pipeline-internal
Flow	<code>CREATE FLOW ... AS INSERT INTO / AUTO CDC INTO</code>	<code>@dp.append_flow,</code> <code>dp.create_auto_cdc_flow</code>	Writes into a streaming table; several flows per table

SQL declarations

```
CREATE OR REFRESH STREAMING TABLE bronze_orders AS
SELECT *, _metadata.file_path AS source_file
FROM STREAM read_files('${order_path}/stream/', format => 'json');

CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_price EXPECT (unit_price >= 0) ON VIOLATION FAIL UPDATE,
  CONSTRAINT has_payment EXPECT (payment_method IS NOT NULL) -- warn only
) AS
SELECT order_id, customer_id, CAST(order_datetime AS TIMESTAMP) AS order_ts,
       quantity * unit_price AS gross_amount
FROM STREAM(bronze_orders);

CREATE OR REFRESH MATERIALIZED VIEW gold_daily_sales AS
SELECT DATE(order_ts) AS order_date, SUM(gross_amount) AS revenue
FROM silver_orders GROUP BY DATE(order_ts);

CREATE OR REFRESH STREAMING TABLE all_orders; -- backfill once, then stream
CREATE FLOW orders_backfill AS INSERT INTO ONCE all_orders BY NAME
  SELECT * FROM read_files('${order_path}/orders_batch.json', format => 'json');
CREATE FLOW orders_stream AS INSERT INTO all_orders BY NAME
  SELECT * FROM STREAM read_files('${order_path}/stream/', format => 'json');
```

Python declarations

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

@dp.table(name="silver_orders")
@dp.expect_or_drop("valid_order_id", "order_id IS NOT NULL")
@dp.expect_or_fail("valid_price", "unit_price >= 0")
@dp.expect("has_payment", "payment_method IS NOT NULL")
def silver_orders():
    return spark.readStream.table("bronze_orders")          # streaming read -> streaming table

@dp.materialized_view(name="gold_daily_sales")
def gold_daily_sales():
    return (spark.read.table("silver_orders")
            .groupBy(F.to_date("order_datetime").alias("order_date"))
            .agg(F.sum("total_amount").alias("revenue")))
```

Expectation	SQL	Python	Invalid rows	Update
Warn (default)	EXPECT (cond)	@dp.expect	Kept, counted	Continues
Drop	ON VIOLATION DROP ROW	@dp.expect_or_drop	Removed, counted	Continues
Fail	ON VIOLATION FAIL UPDATE	@dp.expect_or_fail	None written	Stops

@dp.expect_all, expect_all_or_drop and expect_all_or_fail take a dict of rules. Quarantine pattern: valid rows to silver, rows failing the inverse predicate to a quarantine table.

AUTO CDC

```
CREATE OR REFRESH STREAMING TABLE silver_customers;

CREATE FLOW silver_customers_cdc AS AUTO CDC INTO silver_customers
FROM STREAM(bronze_customers)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY ingestion_ts
COLUMNS * EXCEPT (operation, _rescued_data)
STORED AS SCD TYPE 2;
```

```
dp.create_streaming_table("silver_customers")
dp.create_auto_cdc_flow(target="silver_customers", source="bronze_customers",
                        keys=["customer_id"], sequence_by=F.col("ingestion_ts"),
                        stored_as_scd_type=2)          # 1 = overwrite in place
```

SCD type 1 updates rows in place. SCD type 2 closes the old row and adds __START_AT and __END_AT (current row: __END_AT IS NULL); TRACK HISTORY ON limits which columns create versions. SEQUENCE BY handles out-of-order events. The target must be a streaming table.

Running and monitoring

Setting	Options
Mode	Triggered (process available data, stop) or continuous
Development / production	Development reuses compute and skips retries; production retries
Refresh	Update (incremental) or full refresh (reset checkpoints, rebuild), for all or selected tables
Parameters	Configuration keys: <code>\${key}</code> in SQL, <code>spark.conf.get("key")</code> in Python

```
SELECT timestamp, details:flow_progress.data_quality.expectations
FROM event_log(TABLE(retailhub.lakeflow.silver_orders))    -- or event_log('<pipeline-id>')
WHERE event_type = 'flow_progress'
AND details:flow_progress.data_quality.expectations IS NOT NULL;
```

Legacy (Delta Live Tables)	Current
<code>import dlt</code>	<code>from pyspark import pipelines as dp</code>
<code>@dlt.table (streaming DataFrame)</code>	<code>@dp.table</code>
<code>@dlt.table (batch DataFrame)</code>	<code>@dp.materialized_view</code>
<code>@dlt.view</code>	<code>@dp.temporary_view</code>
<code>TEMPORARY table / temporary=True</code>	<code>PRIVATE table / private=True</code>
<code>CREATE LIVE TABLE, FROM LIVE.x, STREAM(LIVE.x)</code>	<code>CREATE OR REFRESH MATERIALIZED VIEW, FROM x, STREAM(x)</code>
<code>APPLY CHANGES INTO, dlt.apply_changes()</code>	<code>AUTO CDC INTO, dp.create_auto_cdc_flow()</code>

MODULE 08

Lakeflow Jobs

Tasks and control flow

Task type	Runs
Notebook, Python script, wheel, JAR, dbt	Code with parameters
SQL	A query, SQL file or alert on a SQL warehouse
Dashboard	Refreshes an AI/BI dashboard, optionally emails a snapshot
Pipeline	An update of a Lakeflow Spark Declarative Pipeline
Run Job	Another job
If/else condition	Compares two values; downstream tasks follow the true or false outcome
For each	A nested task once per element of a list, with set concurrency

Setting	Detail
Dependencies	depends_on builds the DAG; independent tasks run in parallel
Run if	All succeeded (default), At least one succeeded, None failed, All done, At least one failed, All failed
Statuses	Succeeded, Failed, Timed out, Canceled, Skipped, Upstream failed (not run), Excluded
Retries	Per task: max_retries, min_retry_interval_millis, retry_on_timeout; for transient errors only
Other	Task and job timeouts, notifications (email, webhook, duration warning), max_concurrent_runs

```
tasks:
- task_key: validate
  notebook_task: {notebook_path: ../src/validate}
  max_retries: 2
  min_retry_interval_millis: 60000
- task_key: enough_rows
  depends_on: [{task_key: validate}]
  condition_task: {op: GREATER_THAN, left: "{{tasks.validate.values.row_count}}", right: "100"}
- task_key: check_each_table
  depends_on: [{task_key: enough_rows, outcome: "true"}]
  for_each_task:
    inputs: "{{job.parameters.tables}}"          # JSON array
    concurrency: 2
    task:
      task_key: check_one_table
      notebook_task: {notebook_path: ../src/check, base_parameters: {table: "{{input}}"} }
```

Task values and parameters

```
dbutils.jobs.taskValues.set(key="row_count", value=row_count)           # upstream task
rows = dbutils.jobs.taskValues.get(taskKey="validate", key="row_count",
                                   default=0, debugValue=0)             # downstream task
```

Values must be JSON-serializable, at most 48 KiB. In task settings use `{{tasks.validate.values.row_count}}`, `{{job.parameters.catalog}}`, `{{job.run_id}}`, `{{job.start_time.iso_date}}` and, in For each, `{{input}}`. `dbutils.notebook.exit()` returns a value to `dbutils.notebook.run()`; it does not pass values between job tasks.

Triggers

Trigger	Starts a run when	Settings
Scheduled	A Quartz cron time is reached	Cron, time zone, paused or not
File arrival	New files land in a UC volume or external location	Minimum time between triggers, wait after last change; checked about every minute
Table update	UC tables change (Delta, materialized views, streaming tables, views)	Any or all tables, minimum time between triggers, wait after last change
Continuous	The previous run finishes	One run always active
Manual	Run now, API, CLI	None

Time-based schedules suit fixed deadlines and sources without events. Data-driven triggers (file arrival, table update) suit irregular arrivals: runs start when data is there, without empty runs.

sec	min	hour	day-of-month	month	day-of-week	
0	0	6	*	*	?	every day at 06:00
0	30	2	?	*	MON-FRI	weekdays at 02:30
0	*/15	*	*	*	?	every 15 minutes

Repair run re-executes only selected failed tasks and their downstream tasks, reusing successful results.

MODULE 09

CI/CD with Git folders and bundles

Git folders

Step	Where
Clone the repository	Workspace, Create, Git folder
Create or switch branch, e.g. <code>feature/jan</code>	Git dialog (branch button next to the folder)
Review diff, commit and push	Git dialog: message, Commit & Push
Open, review and merge the pull request	Git provider (GitHub, Azure DevOps, GitLab)
Bring merged changes into the workspace	Git dialog: switch to <code>main</code> , Pull

The Git dialog also merges, rebases, resolves conflicts and resets. After a merge, CI validates and deploys the bundle.

Bundle anatomy

```
# databricks.yml
bundle: {name: retailhub}
include: [resources/*.yaml]           # one file per job, pipeline, ...
variables:
  catalog: {description: Target catalog, default: retailhub_trainer}
targets:
  dev:
    mode: development
    default: true
  prod:
    mode: production
    workspace:
      host: https://adb-1234567890123456.7.azuredatabricks.net
      root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
    variables: {catalog: retailhub_prod}
  run_as: {service_principal_name: "000000000-0000-0000-0000-000000000000"}
```

```
# resources/retailhub_job.yaml
resources:
  jobs:
    retailhub_job:
      name: retailhub_job
      tasks:
        - task_key: run_pipeline
          pipeline_task: {pipeline_id: "${resources.pipelines.retailhub_pipeline.id}"}
```

Relative paths in a resource file resolve from that file. Substitutions include `${var.catalog}`, `${bundle.target}` and `${workspace.current_user.userName}`.

Variable values are taken from, in order of priority: `--var="catalog=retailhub_jan"`, the environment variable `BUNDLE_VAR_catalog`, `.databricks/bundle/<target>/variable-overrides.json`, the target's `variables` block, and finally default.

	mode: development	mode: production
Names	Prefixed [dev <user>]	As declared
Schedules and triggers	Paused	Active
Identity, paths	Deploying user, home folder	Service principal via <code>run_as</code> , shared <code>root_path</code>
Checks	Relaxed: concurrent runs, no lock	Pipelines not in development mode; optional Git branch check

Databricks CLI

```
brew tap databricks/tap && brew install databricks      # macOS, Linux
winget install Databricks.DatabricksCLI                # Windows
databricks auth login --host https://adb-...azuredatabricks.net --profile TRAINING
databricks bundle validate -t dev --var="catalog=retailhub_jan"
databricks bundle deploy -t dev --var="catalog=retailhub_jan"
databricks bundle run -t dev retailhub_job
databricks bundle destroy -t dev
```

The legacy `pip install databricks-cli (0.18)` has no `bundle` command. In CI (e.g. GitHub Actions) install the CLI with `databricks/setup-cli`, authenticate a service principal with `DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET`, then run `bundle validate` and `bundle deploy -t prod`. Promotion changes only target configuration, never the code.

WORKSHOP

Troubleshooting and monitoring

Where to look

Question	First place
When did it start failing or slowing down?	Job Runs tab: run list and matrix view of task durations across runs
Where did it fail?	Run DAG: the first red task; Upstream failed tasks did not run
Why did it fail?	Task output: first line of the error (class, path, table, privilege)
Why is it slow?	Spark UI (classic) or query profile (serverless): stage and operator metrics
Why did it not start?	Compute event log and init script logs
Why did a pipeline drop rows or stop?	Pipeline event log, data quality metrics

Compare runs with their baseline: duration creeping up suggests data growth without maintenance (small files, no clustering); a step change right after a deployment points to that change; failures at the same hour suggest a collision with another workload. Set a duration warning threshold on the job, and query history with SQL:

```
SELECT run_name, result_state, period_start_time,
       timestampdiff(MINUTE, period_start_time, period_end_time) AS minutes
FROM system.lakeflow.job_run_timeline      -- per task: system.lakeflow.job_task_run_timeline
WHERE period_start_time > current_timestamp() - INTERVAL 30 DAYS
ORDER BY period_start_time DESC;
```

Stage metrics

Metric (min, median, max per stage)	Healthy	Problem
Task duration, input size per task	Max close to median	Max far above median: skew
Shuffle read and write	Proportional to data	Very large on joins or aggregations: shuffle
Spill (memory, disk)	0	Above 0: spill
Number of tasks	About data size / 128 MB	Thousands of millisecond tasks: over-partitioned
Join output rows	About input rows	Many times input: wrong join key

Failure patterns

Failure	Symptom	Look at	Fix
Executor OOM	Task retries, <code>OutOfMemoryError</code> , lost executors	Spill, partition sizes, broadcast of a large table	More shuffle partitions, fix skew, lower broadcast threshold, larger workers
Driver OOM	Notebook detaches after <code>collect()</code> or <code>toPandas()</code>	Driver log, collecting cell	Write to a table, <code>limit()</code> , larger driver
Library conflict	<code>ModuleNotFoundError</code> , <code>AttributeError</code> , pip resolver errors	Compute event log, library status	Pin versions; notebook-scoped <code>%pip</code> ; per-job environments
Cluster start failure	Run fails with no task time: init script, capacity, quota	Compute event log, init script logs	Fix the init script; other node type or zone; pools; serverless
Permission denied	<code>PERMISSION_DENIED</code> ... <code>SELECT</code> on table	Securable named in the error	Grant it plus <code>USE CATALOG</code> and <code>USE SCHEMA</code>
Not found	<code>TABLE_OR_VIEW_NOT_FOUND</code> , <code>PATH_NOT_FOUND</code>	First error line, <code>SHOW TABLES</code>	Correct the name; add the missing <code>depends_on</code>
Silent NULL columns	Load succeeds, columns empty	Schema vs JSON names or CSV column order	Fix names (JSON) or order (CSV)

Rule of thumb: no task time means infrastructure; an error before any stage means analysis or permissions; stages ran means performance or data.

MODULE 10

Governance and security

Privileges

Privilege	On	Allows
USE CATALOG, USE SCHEMA	Catalog, schema	Reaching objects inside; needed in addition to object privileges
SELECT, MODIFY	Table, view (or a schema or catalog for all of them)	Read; insert, update, delete
CREATE SCHEMA, CREATE TABLE, CREATE VOLUME, CREATE FUNCTION	Catalog, schema	Create objects
READ VOLUME, WRITE VOLUME, EXECUTE	Volume, function	Files; calling a function
READ FILES, WRITE FILES, CREATE EXTERNAL TABLE	External location	Direct file access, external tables
MANAGE	Any securable	Grant, revoke, drop without ownership
ALL PRIVILEGES	Any securable	All applicable privileges except MANAGE

Unity Catalog is deny by default. Privileges on a catalog or schema are **inherited** by current and future objects inside. Grants can be made by the owner, the owner of the parent catalog or schema, a holder of **MANAGE**, or a metastore admin. Grant to **account groups** (workspace-local groups are rejected) and to service principals (by application ID) for automation.

```
GRANT USE CATALOG ON CATALOG retailhub TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA retailhub.gold TO `analysts`;
GRANT SELECT ON TABLE retailhub.bronze.customers TO `analysts`;
REVOKE SELECT ON TABLE retailhub.bronze.customers FROM `analysts`;
SHOW GRANTS ON SCHEMA retailhub.gold;
ALTER TABLE retailhub.gold.daily_sales OWNER TO `data_engineers`;
```

UI: **Catalog Explorer**, object, **Permissions**, **Grant** or **Revoke**.

DENY: the SQL **DENY** statement works only on the legacy `hive_metastore` catalog, where it overrides grants. Unity Catalog has no **DENY** on data privileges: privileges are additive, so remove access with **REVOKE** at the level where it was granted, grant at a finer level instead of the whole schema, or restrict data with row filters, column masks and ABAC. ABAC **DENY** policies (Beta) can deny only **MANAGE ACCESS CONTROL**.

Row filters and column masks

```
CREATE OR REPLACE FUNCTION retailhub.gold.mask_email(email STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('admins') THEN email
             ELSE concat('***@', substring_index(email, '@', -1)) END;
ALTER TABLE retailhub.bronze.customers ALTER COLUMN email SET MASK retailhub.gold.mask_email;

CREATE OR REPLACE FUNCTION retailhub.bronze.segment_filter(customer_segment STRING)
RETURNS BOOLEAN
RETURN is_account_group_member('admins') OR customer_segment = 'Premium';
ALTER TABLE retailhub.bronze.customers
  SET ROW FILTER retailhub.bronze.segment_filter ON (customer_segment);

ALTER TABLE retailhub.bronze.customers ALTER COLUMN email DROP MASK;
ALTER TABLE retailhub.bronze.customers DROP ROW FILTER;
```

A row filter function returns **BOOLEAN** (the row is visible when true); a mask returns the column's type. Both apply to every query on the table.

Attribute-based access control (ABAC)

Stewards tag tables and columns with **governed tags**; a **policy** on a catalog, schema or table applies a filter or mask function to every object whose tags match.

```
ALTER TABLE retailhub.bronze.customers ALTER COLUMN email SET TAGS ('pii' = 'email');

CREATE OR REPLACE POLICY mask_pii_email
ON SCHEMA retailhub.bronze
COLUMN MASK retailhub.gold.mask_email
TO `account users` EXCEPT `admins`
FOR TABLES
MATCH COLUMNS has_tag_value('pii', 'email') AS email_col
ON COLUMN email_col;

CREATE OR REPLACE POLICY premium_only
ON SCHEMA retailhub.bronze
ROW FILTER retailhub.bronze.segment_filter
TO `analysts`
FOR TABLES
WHEN has_tag_value('sensitivity', 'high')
MATCH COLUMNS has_tag('segment') AS seg
USING COLUMNS (seg);

SHOW POLICIES ON SCHEMA retailhub.bronze;
DROP POLICY mask_pii_email ON SCHEMA retailhub.bronze;
```

The camelCase `hasTagValue` and `hasTag` still work. A policy requires **MANAGE** (or ownership) on the securable and **EXECUTE** on the function.

Managed and external tables

```
CREATE TABLE retailhub.silver.orders (order_id STRING, amount DOUBLE);           -- managed
CREATE TABLE retailhub.bronze.orders_ext (order_id STRING, amount DOUBLE)       -- external
LOCATION 'abfss://landing@retailhubadls.dfs.core.windows.net/orders/';

ALTER TABLE retailhub.bronze.orders_ext SET MANAGED;                          -- external to managed
ALTER TABLE retailhub.bronze.orders_ext UNSET MANAGED;                        -- roll back within 14 days
UNDROP TABLE retailhub.silver.orders;                                         -- within the recovery period (7 days)
```

Action	Managed	External
Create	No LOCATION	LOCATION inside an external location; needs CREATE EXTERNAL TABLE
Modify	DML and ALTER TABLE; maintained by predictive optimization	Same SQL; maintenance is yours
Drop	Metadata and data removed	Metadata removed; files stay
Convert	To external: no in-place command; recreate with DEEP CLONE ... LOCATION or CTAS with LOCATION	To managed: SET MANAGED (Delta, table owner, DBR 17.3 LTS+ or serverless)

External access is built from a **storage credential** (cloud identity, such as an Azure access connector) and an **external location** (a path plus that credential).

APPENDIX A

Exam at a glance

Item	Value
Exam guide	Databricks Certified Data Engineer Associate, 4 May 2026
Format	45 scored multiple-choice questions (unscored items may be added), 90 minutes
Passing score	Not published; aim for at least 80% on practice tests
Fee, validity	USD 200; 2 years
Code	Mostly PySpark; SQL for ingestion, DDL and governance
Experience	About six months of hands-on Databricks data engineering

#	Domain	Weight	Modules
1	Databricks Intelligence Platform	6%	01
2	Data Ingestion and Loading	21%	02, 02a, 05
3	Data Transformation and Modeling	22%	03, 05a, 06, 07
4	Working with Lakeflow Jobs	16%	08
5	Implementing CI/CD	10%	09
6	Troubleshooting, Monitoring, and Optimization	10%	04, troubleshooting workshop
7	Governance and Security	15%	10

Frequent traps

Topic	Remember
Expectation without <code>ON VIOLATION</code>	Warn only: rows kept, metrics recorded
PySpark union	Like SQL <code>UNION ALL</code> : by position, duplicates kept
CSV with a user schema	Positional; header names ignored
<code>COPY INTO</code> rerun	Loaded files skipped unless <code>COPY_OPTIONS ('force' = 'true')</code>
<code>OPTIMIZE</code> vs <code>VACUUM</code>	Compacts files vs deletes unreferenced files (7-day default retention)
Liquid clustering	Up to 4 keys; not with partitioning or Z-order; <code>CLUSTER BY AUTO</code> on managed tables
Predictive optimization	Unity Catalog managed tables only
Repair run	Reruns failed tasks and their downstream tasks only
Values between tasks	<code>dbutils.jobs.taskValues</code> , not <code>notebook.exit</code>
Job schedule	Quartz cron with seconds: <code>0 0 6 * * ?</code>
<code>mode: development</code>	<code>[dev user]</code> prefix, paused triggers
<code>DENY</code>	Not in Unity Catalog; use <code>REVOKE</code> and finer grants
Serverless	<code>availableNow</code> , query profile, volumes; no <code>processingTime</code> , no Spark UI
Skew	Max task time far above median; AQE skew join